Compiled PDF
Project: CODES

Project Structure: CODES

==========

```
|-- qp1_merging_transformation.py
|-- qp2_import_transform_stats.py
|-- qp3_cleaning_univariate.py
|-- qp4_bivariate.py
|-- qp5_multivariate.py
|-- qp6_time_series.py
|-- qp7_1d_2d_nd_analysis.py
|-- qp8_clustering.py
|-- qp9_pca.py
|__ requirements.txt
```

(10 files total)

File: c:\Users\HP\OneDrive\Desktop\Computer_Science\EDA\CODES\qp1_merging_transformation.py

```
import numpy as np
import pandas as pd

pd.set_option("display.width", 200)


# PART A : DATAFRAME OPERATIONS

dataframe1 = pd.DataFrame({
    "ProductID":  [101, 102, 103, 104, 105, 106, 107, 108, 109, 110],
    "NameofProd": ["Laptop", "Mouse", "Keyboard", "Monitor", "Printer",
                   "Scanner", "Webcam", "Headset", "Router", "Tablet"],
    "Price":      [55000, 500, 1200, 9000, 7500, 6500, 2500, 3000, 2200, 18000],
})

dataframe2 = pd.DataFrame({
    "ProductID":        [105, 106, 107, 108, 109, 110, 111, 112, 113, 114],
    "Company_Location": ["Chennai", "Mumbai", "Delhi", "Pune", "Bengaluru",
                         "Hyderabad", "Kolkata", "Jaipur", "Kochi", "Noida"],
})

dataframe3 = pd.DataFrame({
    "ProductID":  [111, 112, 113, 114, 115, 116, 117, 118, 119, 120],
    "NameofProd": ["Speaker", "Pen Drive", "Hard Disk", "SSD", "Projector",
                   "Smart Watch", "Power Bank", "Charger", "HDMI Cable", "UPS"],
    "Price":      [1800, 600, 4200, 5200, 32000, 8000, 1500, 900, 350, 4800],
})

print("dataframe1\n", dataframe1, "\n")
print("dataframe2\n", dataframe2, "\n")
print("dataframe3\n", dataframe3, "\n")

# 1. Append the records of dataframe1 and dataframe3 (row-wise)
appended = pd.concat([dataframe1, dataframe3], ignore_index=True)
print("1. APPEND dataframe1 + dataframe3 (20 rows)\n", appended, "\n")

# 2. Concatenate wherever applicable
concat_rows = pd.concat([dataframe1, dataframe3], axis=0, ignore_index=True)
concat_cols = pd.concat([dataframe1, dataframe2], axis=1)
print("2a. CONCAT rows (axis=0) df1 & df3\n", concat_rows.head(), "\n")
print("2b. CONCAT columns (axis=1) df1 & df2\n", concat_cols, "\n")

# 3. Merge dataframe1 and dataframe2 using ProductID
merged = pd.merge(dataframe1, dataframe2, on="ProductID")
print("3. MERGE on ProductID\n", merged, "\n")

# 4. Join operations between dataframe1 and dataframe2
inner_join = pd.merge(dataframe1, dataframe2, on="ProductID", how="inner")
```

File: c:\Users\HP\OneDrive\Desktop\Computer_Science\EDA\CODES\qp1_merging_transformation.py

```
49 | left_join = pd.merge(dataframe1, dataframe2, on="ProductID", how="left")
50 | right_join = pd.merge(dataframe1, dataframe2, on="ProductID", how="right")
51 | outer_join = pd.merge(dataframe1, dataframe2, on="ProductID", how="outer")
52 |
53 | print("4a. INNER JOIN\n", inner_join, "\n")
54 | print("4b. LEFT JOIN\n", left_join, "\n")
55 | print("4c. RIGHT JOIN\n", right_join, "\n")
56 | print("4d. FULL OUTER JOIN\n", outer_join, "\n")
57 |
58 | # PART B : DATA TRANSFORMATIONS
59 |
60 | REG_LAST4 = 287
61 | Yuktha = np.array([REG_LAST4, 45, 12, 78, 33, 90, 21, 67, 54, 8], dtype=float)
62 |
63 | print("Vector 'Yuktha':", Yuktha, "\n")
64 |
65 |
66 | def rescale(x, new_min, new_max):
67 |     return new_min + (x - x.min()) * (new_max - new_min) / (x.max() - x.min())
68 |
69 |
70 | # 1. Normalisation to given ranges
71 | norm_m3_p3 = rescale(Yuktha, -3, 3)
72 | norm_m1_p1 = rescale(Yuktha, -1, 1)
73 | norm_0_1 = rescale(Yuktha, 0, 1)          # classic Min-Max normalisation
74 |
75 | print("1a. Normalised to [-3, +3]:\n", np.round(norm_m3_p3, 4))
76 | print("1b. Normalised to [-1, +1]:\n", np.round(norm_m1_p1, 4))
77 | print("1c. Min-Max [0, 1]:\n", np.round(norm_0_1, 4), "\n")
78 |
79 | # 2. Z-score (sample standard deviation, same as R's scale())
80 | z_score = (Yuktha - Yuktha.mean()) / Yuktha.std(ddof=1)
81 | print("2. Z-score:\n", np.round(z_score, 4), "\n")
82 |
83 | # 3. Decimal scaling:  v' = v / 10^j,  j = smallest integer with max(|v'|) < 1
84 | j = int(np.floor(np.log10(np.abs(Yuktha).max()))) + 1
85 | decimal_scaled = Yuktha / (10 ** j)
86 | print(f"3. Decimal scaling (j = {j}):\n", np.round(decimal_scaled, 4))
87 |
```

File: c:\Users\HP\OneDrive\Desktop\Computer_Science\EDA\CODES\qp2_import_transform_stats.py

```
import numpy as np
import pandas as pd

pd.set_option("display.width", 200)
pd.set_option("display.max_columns", 50)

URL = "https://raw.githubusercontent.com/salemprakash/EDA/main/Data/Employee%20Attrition.csv"
INJECT_DEMO_DATA = False

# 1. Import the dataset
df = pd.read_csv(URL)
print("1. Dataset imported.\n")

if INJECT_DEMO_DATA:
    rng = np.random.default_rng(42)
    df = df.copy()
    df.loc[rng.choice(df.index, 60, replace=False), "Age"] = np.nan
    df.loc[rng.choice(df.index, 80, replace=False), "MonthlyIncome"] = np.nan
    df.loc[rng.choice(df.index, 50, replace=False), "Department"] = np.nan
    df.loc[rng.choice(df.index, 40, replace=False), "YearsAtCompany"] = np.nan
    df = pd.concat([df, df.sample(10, random_state=1)], ignore_index=True)  # 10 duplicates

# 2. First 10 records
print("2. First 10 records\n", df.head(10), "\n")

# 3. Last 10 records
print("3. Last 10 records\n", df.tail(10), "\n")

# 4. Rows and columns
print(f"4. Rows = {df.shape[0]}, Columns = {df.shape[1]}\n")

# 5. Column names
print("5. Column names:\n", list(df.columns), "\n")

# 6. Data types / structure
print("6. Data types")
df.info()
print()

# 7. Summary statistics
print("7. Summary statistics\n", df.describe(include="all").T.head(15), "\n")

# 8 & 10. Missing values in each column
print("8/10. Missing values per column\n", df.isna().sum()[df.isna().sum() > 0], "\n")

# 9. Total missing values
print("9. Total missing values:", int(df.isna().sum().sum()), "\n")

```

File: c:\Users\HP\OneDrive\Desktop\Computer_Science\EDA\CODES\qp2_import_transform_stats.py

```
# 11. Rows with at least one missing value
rows_with_na = df[df.isna().any(axis=1)]
print(f"11. Rows containing at least one missing value: {len(rows_with_na)}")
print(rows_with_na.head(), "\n")

# 12. Remove all rows containing missing values
df_dropna = df.dropna()
print(f"12. After dropping rows with NaN: {df_dropna.shape}")
print(df_dropna.head(), "\n")

# 13. Replace missing values in a numerical column with the MEAN
df_fill = df.copy()
df_fill["Age"] = df_fill["Age"].fillna(df_fill["Age"].mean())
print("13. Age NaN after mean fill:", df_fill["Age"].isna().sum())

# 14. Replace missing values in another numerical column with the MEDIAN
df_fill["MonthlyIncome"] = df_fill["MonthlyIncome"].fillna(df_fill["MonthlyIncome"].median())
print("14. MonthlyIncome NaN after median fill:", df_fill["MonthlyIncome"].isna().sum())

# 15. Replace missing values in a categorical column with the MODE
df_fill["Department"] = df_fill["Department"].fillna(df_fill["Department"].mode()[0])
print("15. Department NaN after mode fill:", df_fill["Department"].isna().sum(), "\n")

# 16. Forward fill  (equivalent of zoo::na.locf in R)
df_ffill = df.copy()
df_ffill["YearsAtCompany"] = df_ffill["YearsAtCompany"].ffill()
print("16. YearsAtCompany NaN after forward fill:", df_ffill["YearsAtCompany"].isna().sum())

# 17. Backward fill
df_bfill = df.copy()
df_bfill["YearsAtCompany"] = df_bfill["YearsAtCompany"].bfill()
print("17. YearsAtCompany NaN after backward fill:", df_bfill["YearsAtCompany"].isna().sum())

# 18. Replace remaining missing values with a constant (0 / "Unknown")
df_const = df.copy()
num_cols = df_const.select_dtypes(include="number").columns
cat_cols = df_const.select_dtypes(exclude="number").columns
df_const[num_cols] = df_const[num_cols].fillna(0)
df_const[cat_cols] = df_const[cat_cols].fillna("Unknown")
print("18. Remaining NaN after constant fill:", int(df_const.isna().sum().sum()), "\n")

# 19. Remove duplicate records
print("19. Duplicate rows found:", df_fill.duplicated().sum())
df_nodup = df_fill.drop_duplicates()
print("    Shape after removing duplicates:", df_nodup.shape, "\n")

# 20. Convert appropriate variables into factors (category dtype)
df_clean = df_nodup.copy()
```

File: c:\Users\HP\OneDrive\Desktop\Computer_Science\EDA\CODES\qp2_import_transform_stats.py

```
97 | factor_cols = ["Attrition", "BusinessTravel", "Department", "EducationField", "Gender",
98 |                "JobRole", "MaritalStatus", "Over18", "OverTime"]
99 | df_clean[factor_cols] = df_clean[factor_cols].astype("category")
100 | print("20. Dtypes after converting to category:\n", df_clean[factor_cols].dtypes, "\n")
101 |
102 | # 21. Rename column names
103 | df_clean = df_clean.rename(columns={"MonthlyIncome": "Monthly_Income",
104 |                                     "DistanceFromHome": "Distance_From_Home"})
105 | print("21. Renamed columns ->", [c for c in df_clean.columns if "_" in c], "\n")
106 |
107 | # 22. New derived column
108 | df_clean["Annual_Income"] = df_clean["Monthly_Income"] * 12
109 | print("22. Derived column Annual_Income\n", df_clean[["Monthly_Income", "Annual_Income"]].head(), "\n")
110 |
111 | # 23. summary() equivalent
112 | print("23. Summary statistics\n", df_clean.describe().T.head(12), "\n")
113 |
114 | # 24. Min and max
115 | print("24. Min / Max of Monthly_Income:", df_clean["Monthly_Income"].min(),
116 |       "/", df_clean["Monthly_Income"].max())
117 |
118 | # 25. Range
119 | print("25. Range of Monthly_Income:",
120 |       df_clean["Monthly_Income"].max() - df_clean["Monthly_Income"].min())
121 |
122 | # 26. Variance
123 | print("26. Variance of Monthly_Income:", round(df_clean["Monthly_Income"].var(), 2))
124 |
125 | # 27. Standard deviation
126 | print("27. Std. deviation of Monthly_Income:", round(df_clean["Monthly_Income"].std(), 2))
127 |
128 | # 28. IQR
129 | q1, q3 = df_clean["Monthly_Income"].quantile([0.25, 0.75])
130 | print("28. IQR of Monthly_Income:", q3 - q1)
131 |
```

File: c:\Users\HP\OneDrive\Desktop\Computer_Science\EDA\CODES\qp3_cleaning_univariate.py

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

pd.set_option("display.width", 200)
pd.set_option("display.max_columns", 50)
sns.set_theme(style="whitegrid")

URL = "https://raw.githubusercontent.com/salemprakash/EDA/main/Data/Employee%20Attrition.csv"

# 1. Import
df = pd.read_csv(URL)

"""# demo corruption
rng = np.random.default_rng(7)
df.loc[rng.choice(df.index, 70, replace=False), "Age"] = np.nan
df.loc[rng.choice(df.index, 90, replace=False), "MonthlyIncome"] = np.nan
df.loc[rng.choice(df.index, 60, replace=False), "Department"] = np.nan
df["Sparse_Col"] = np.where(rng.random(len(df)) < 0.80, np.nan, 1.0)   # ~80 % missing
df = pd.concat([df, df.sample(12, random_state=3)], ignore_index=True)  # 12 duplicates"""


print("1. Dataset imported:", df.shape, "\n")

# 2. First and last six records
print("2. First six\n", df.head(6), "\n\n   Last six\n", df.tail(6), "\n")

# 3. Total missing values
print("3. Total missing values:", int(df.isna().sum().sum()), "\n")

# 4. Records with at least one missing value
print("4. Records with >= 1 missing value:", df.isna().any(axis=1).sum())
print(df[df.isna().any(axis=1)].head(), "\n")

# 5. Remove columns having > 70 % missing values
pct_missing = df.isna().mean() * 100
drop_cols = pct_missing[pct_missing > 70].index.tolist()
df = df.drop(columns=drop_cols)
print("5. Columns dropped (>70 % missing):", drop_cols, "\n")

# 6. Replace missing numerical values with the MEAN   (Age)
df["Age"] = df["Age"].fillna(df["Age"].mean())
print("6. Age missing after mean imputation:", df["Age"].isna().sum())

# 7. Replace missing numerical values with the MEDIAN (MonthlyIncome)
df["MonthlyIncome"] = df["MonthlyIncome"].fillna(df["MonthlyIncome"].median())
```

File: c:\Users\HP\OneDrive\Desktop\Computer_Science\EDA\CODES\qp3_cleaning_univariate.py

```
print("7. MonthlyIncome missing after median imputation:", df["MonthlyIncome"].isna().sum())

# 8. Replace missing categorical values with the MODE (Department)
df["Department"] = df["Department"].fillna(df["Department"].mode()[0])
print("8. Department missing after mode imputation:", df["Department"].isna().sum(), "\n")

# 9. Identify duplicate rows
dups = df[df.duplicated(keep=False)]
print("9. Duplicate rows (all copies):", len(dups), "| extra copies:", df.duplicated().sum())

# 10. Remove duplicate rows
df = df.drop_duplicates().reset_index(drop=True)
print("10. Shape after dropping duplicates:", df.shape, "\n")

# 11. Columns with incorrect data types
print("11. Columns with questionable data types")
print("    object columns that should be categorical:",
      df.select_dtypes(exclude="number").columns.tolist())
ordinal_like = [c for c in df.select_dtypes("number").columns if df[c].nunique() <= 5]
print("    integer columns with <=5 levels (really categorical/ordinal):", ordinal_like, "\n")

# 12. Convert character variables into factors (category)
for c in df.select_dtypes(exclude=["number", "category"]).columns:
    df[c] = df[c].astype("category")
print("12. Dtypes after conversion\n", df.dtypes.value_counts(), "\n")

# 13. Convert text values to lower / upper case
df["Department_lower"] = df["Department"].astype(str).str.lower()
df["Gender_upper"] = df["Gender"].astype(str).str.upper()
print("13. Case conversion\n", df[["Department", "Department_lower", "Gender", "Gender_upper"]].head(), "\n")

# 14. Detect outliers using boxplots
num_for_box = ["MonthlyIncome", "TotalWorkingYears", "YearsAtCompany", "YearsSinceLastPromotion"]
fig, axes = plt.subplots(1, 4, figsize=(15, 5))
for ax, col in zip(axes, num_for_box):
    sns.boxplot(y=df[col], ax=ax, color="skyblue")
    ax.set_title(col)
fig.suptitle("14. Boxplots for outlier detection")
plt.tight_layout(); plt.show()

# 15. Identify outliers with the IQR method
def iqr_bounds(s, k=1.5):
    q1, q3 = s.quantile([0.25, 0.75])
    iqr = q3 - q1
    return q1 - k * iqr, q3 + k * iqr

print("15. IQR outliers (1.5 x IQR)")
for col in num_for_box:
```

File: c:\Users\HP\OneDrive\Desktop\Computer_Science\EDA\CODES\qp3_cleaning_univariate.py

```
    lo, hi = iqr_bounds(df[col])
    n = ((df[col] < lo) | (df[col] > hi)).sum()
    print(f"    {col:25s} bounds=({lo:9.1f}, {hi:9.1f})  outliers={n}")
print()

# 16. Remove extreme outliers (3 x IQR) on MonthlyIncome
lo, hi = iqr_bounds(df["MonthlyIncome"], k=3)
before = len(df)
df_noext = df[(df["MonthlyIncome"] >= lo) & (df["MonthlyIncome"] <= hi)]
print(f"16. Extreme outliers removed: {before - len(df_noext)}  -> shape {df_noext.shape}\n")

# 17. Histogram + interpretation
plt.figure(figsize=(8, 5))
plt.hist(df["MonthlyIncome"], bins=30, color="steelblue", edgecolor="white")
plt.title("17. Histogram of MonthlyIncome"); plt.xlabel("MonthlyIncome"); plt.ylabel("Frequency")
plt.show()
sk = stats.skew(df["MonthlyIncome"])
print(f"17. Interpretation: skewness = {sk:.2f}; the distribution is "
      f"{'right (positively) skewed' if sk > 0.5 else 'left skewed' if sk < -0.5 else 'roughly symmetric'}"
      f" - most employees earn modest salaries while a few earn very high ones.\n")

# 18. Histogram with density curve
plt.figure(figsize=(8, 5))
sns.histplot(df["MonthlyIncome"], bins=30, stat="density", kde=True, color="steelblue")
plt.title("18. Histogram with density curve")
plt.show()

# 19. Violin plot
plt.figure(figsize=(8, 5))
sns.violinplot(y=df["MonthlyIncome"], color="lightgreen")
plt.title("19. Violin plot of MonthlyIncome")
plt.show()

# 20. Dot plot for a small numerical dataset
small = df["Age"].sample(30, random_state=1).sort_values().reset_index(drop=True)
plt.figure(figsize=(8, 5))
sns.stripplot(x=small, size=8, color="crimson", jitter=0.15)
plt.title("20. Dot plot of 30 sampled ages"); plt.xlabel("Age")
plt.show()

# 21. Frequency table of a categorical variable
freq = df["JobRole"].value_counts()
print("21. Frequency table (JobRole)\n", freq, "\n")

# 22. Proportion and percentage
prop = df["JobRole"].value_counts(normalize=True)
print("22. Proportion & percentage\n",
      pd.DataFrame({"Count": freq, "Proportion": prop.round(4), "Percent": (prop * 100).round(2)}), "\n")
```

File: c:\Users\HP\OneDrive\Desktop\Computer_Science\EDA\CODES\qp3_cleaning_univariate.py

```
145 |
146 | # 23. Bar chart of frequencies
147 | plt.figure(figsize=(9, 5))
148 | sns.countplot(data=df, x="Department", hue="Department", palette="Set2", legend=False)
149 | plt.title("23. Frequency of each Department"); plt.ylabel("Count")
150 | plt.show()
151 |
152 | # 24. Horizontal bar chart (several categories)
153 | plt.figure(figsize=(9, 5))
154 | freq.sort_values().plot(kind="barh", color="teal")
155 | plt.title("24. JobRole frequency (horizontal)"); plt.xlabel("Count")
156 | plt.tight_layout(); plt.show()
157 |
158 | # 25. Percentage bar chart
159 | plt.figure(figsize=(9, 5))
160 | (prop.sort_values() * 100).plot(kind="barh", color="orange")
161 | plt.title("25. JobRole percentage"); plt.xlabel("Percentage (%)")
162 | plt.tight_layout(); plt.show()
163 |
164 | # 26. Pie chart (small number of categories)
165 | plt.figure(figsize=(6, 6))
166 | df["Attrition"].value_counts().plot(kind="pie", autopct="%1.1f%%", colors=["#66b3ff", "#ff9999"], startangle=90)
167 | plt.title("26. Attrition share"); plt.ylabel("")
168 | plt.show()
169 |
170 | # 27. Lollipop chart
171 | f = freq.sort_values()
172 | plt.figure(figsize=(9, 5))
173 | plt.hlines(y=f.index, xmin=0, xmax=f.values, color="grey")
174 | plt.plot(f.values, f.index, "o", color="purple", markersize=9)
175 | plt.title("27. Lollipop chart - JobRole"); plt.xlabel("Count")
176 | plt.tight_layout(); plt.show()
177 |
178 | # 28. Dot chart (Cleveland dot plot)
179 | plt.figure(figsize=(9, 5))
180 | plt.scatter(f.values, f.index, color="darkgreen", s=60)
181 | plt.grid(axis="y", linestyle=":")
182 | plt.title("28. Dot chart - JobRole counts"); plt.xlabel("Count")
183 | plt.tight_layout(); plt.show()
184 |
185 | # 29. Density plot
186 | plt.figure(figsize=(8, 5))
187 | sns.kdeplot(df["Age"], fill=True, color="royalblue")
188 | plt.title("29. Density plot of Age")
189 | plt.show()
190 |
191 | # 30. facet_wrap equivalent - distribution of one variable across groups
192 | g = sns.displot(data=df, x="MonthlyIncome", col="Department", col_wrap=3,
```

File: c:\Users\HP\OneDrive\Desktop\Computer_Science\EDA\CODES\qp3_cleaning_univariate.py

```
193 |                 kde=True, bins=25, height=4, facet_kws={"sharey": False})
194 | g.figure.suptitle("30. MonthlyIncome distribution by Department", y=1.03)
195 | plt.show()
196 |
197 | print("30. Interpretation per group")
198 | for dept, sub in df.groupby("Department", observed=True)["MonthlyIncome"]:
199 |     print(f"    {dept:25s} mean={sub.mean():8.1f} median={sub.median():8.1f} skew={stats.skew(sub):5.2f}"
200 |           f" -> {'right-skewed' if stats.skew(sub) > 0.5 else 'fairly symmetric'}")
201 |
```

File: c:\Users\HP\OneDrive\Desktop\Computer_Science\EDA\CODES\qp4_bivariate.py

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")
URL = "https://raw.githubusercontent.com/salemprakash/EDA/main/Data/Employee%20Attrition.csv"

# 1. Load dataset
df = pd.read_csv(URL)
print("1. Loaded:", df.shape, "\n")

# 2. Two categorical variables -> contingency table + stacked bar plots
ct = pd.crosstab(df["Department"], df["Attrition"])
print("2. Contingency table (Department vs Attrition)\n", ct, "\n")

# 2a. Simple stacked bar plot (basic matplotlib/pandas  ~ Base R)
ct.plot(kind="bar", stacked=True, figsize=(8, 5), color=["#4c72b0", "#dd8452"])
plt.title("2a. Stacked Bar (matplotlib): Department vs Attrition")
plt.xlabel("Department"); plt.ylabel("Number of Employees")
plt.xticks(rotation=15); plt.legend(title="Attrition")
plt.tight_layout(); plt.show()

# 2b. Stacked bar plot with seaborn (~ ggplot2)
plt.figure(figsize=(8, 5))
sns.histplot(data=df, x="Department", hue="Attrition", multiple="stack", shrink=0.8,
             palette="Set2")
plt.title("2b. Stacked Bar (seaborn): Department vs Attrition")
plt.ylabel("Number of Employees")
plt.tight_layout(); plt.show()

# 3. Grouped / side-by-side bar plot
plt.figure(figsize=(9, 5))
sns.countplot(data=df, x="MaritalStatus", hue="Attrition", palette="Set1")
plt.title("3. Grouped Bar: MaritalStatus vs Attrition")
plt.xlabel("Marital Status"); plt.ylabel("Count"); plt.legend(title="Attrition")
plt.tight_layout(); plt.show()

# 4. Scatter plot of two numerical variables
plt.figure(figsize=(8, 6))
plt.scatter(df["Age"], df["MonthlyIncome"], alpha=0.5, color="teal", edgecolor="white")
plt.title("4. Scatter: Age vs MonthlyIncome")
plt.xlabel("Age"); plt.ylabel("Monthly Income")
plt.tight_layout(); plt.show()

# 5. Line plot (trend): average income by years at company
trend = df.groupby("YearsAtCompany")["MonthlyIncome"].mean()
plt.figure(figsize=(9, 5))
```

File: c:\Users\HP\OneDrive\Desktop\Computer_Science\EDA\CODES\qp4_bivariate.py

```
plt.plot(trend.index, trend.values, marker="o", color="crimson", linewidth=2)
plt.title("5. Line Plot: Mean MonthlyIncome by YearsAtCompany")
plt.xlabel("Years at Company"); plt.ylabel("Mean Monthly Income")
plt.grid(True, linestyle="--", alpha=0.6)
plt.tight_layout(); plt.show()

# 6. Heatmap of relationship among variables
num_cols = ["Age", "DailyRate", "DistanceFromHome", "HourlyRate", "MonthlyIncome",
            "TotalWorkingYears", "YearsAtCompany", "YearsInCurrentRole",
            "YearsWithCurrManager", "PercentSalaryHike"]
corr = df[num_cols].corr()
plt.figure(figsize=(10, 8))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", linewidths=0.5, cbar_kws={"label": "Correlation"})
plt.title("6. Heatmap of Numerical Variables")
plt.tight_layout(); plt.show()

# 7. Bar chart comparing values (mean income by department)
mean_inc = df.groupby("Department")["MonthlyIncome"].mean().sort_values()
plt.figure(figsize=(8, 5))
bars = plt.bar(mean_inc.index, mean_inc.values, color=["#8da0cb", "#66c2a5", "#fc8d62"])
plt.bar_label(bars, fmt="%.0f")
plt.title("7. Bar Chart: Mean MonthlyIncome by Department")
plt.xlabel("Department"); plt.ylabel("Mean Monthly Income")
plt.tight_layout(); plt.show()

# 8. Density plot: numerical across categories
plt.figure(figsize=(8, 5))
sns.kdeplot(data=df, x="Age", hue="Attrition", fill=True, common_norm=False, alpha=0.4)
plt.title("8. Density Plot: Age by Attrition")
plt.tight_layout(); plt.show()

# 9. Box plot
plt.figure(figsize=(8, 5))
sns.boxplot(data=df, x="Department", y="MonthlyIncome", hue="Department", palette="Set3", legend=False)
plt.title("9. Box Plot: MonthlyIncome by Department")
plt.tight_layout(); plt.show()

# 10. Violin plot
plt.figure(figsize=(8, 5))
sns.violinplot(data=df, x="Department", y="MonthlyIncome", hue="Department", palette="Pastel1", legend=False)
plt.title("10. Violin Plot: MonthlyIncome by Department")
plt.tight_layout(); plt.show()

# 11. Violin + box in one graphic
plt.figure(figsize=(8, 5))
sns.violinplot(data=df, x="JobSatisfaction", y="MonthlyIncome", hue="JobSatisfaction",
               palette="muted", inner=None, legend=False)
sns.boxplot(data=df, x="JobSatisfaction", y="MonthlyIncome", width=0.15,
```

File: c:\Users\HP\OneDrive\Desktop\Computer_Science\EDA\CODES\qp4_bivariate.py

```
97 |             boxprops={"facecolor": "white", "zorder": 3}, showfliers=False)
98 | plt.title("11. Violin + Box: MonthlyIncome by JobSatisfaction")
99 | plt.tight_layout(); plt.show()
100 |
101 | # 12. Correlation matrix + correlogram (lower triangle)
102 | corr_matrix = df[num_cols].corr()
103 | print("12. Correlation matrix\n", corr_matrix.round(2), "\n")
104 | mask = np.triu(np.ones_like(corr_matrix, dtype=bool))
105 | plt.figure(figsize=(10, 8))
106 | sns.heatmap(corr_matrix, mask=mask, annot=True, fmt=".2f", cmap="RdBu_r", vmin=-1, vmax=1,
107 |             square=True, linewidths=0.5, cbar_kws={"label": "Pearson r"})
108 | plt.title("12. Correlogram of Numerical Variables")
109 | plt.tight_layout(); plt.show()
110 |
111 | # 13. Titles, axis labels, legends and parameters are set on every plot above.
112 | print("13. All plots include titles, axis labels, legends and graphical parameters.")
113 |
```

File: c:\Users\HP\OneDrive\Desktop\Computer_Science\EDA\CODES\qp5_multivariate.py

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from pandas.plotting import parallel_coordinates

sns.set_theme(style="whitegrid")
URL = "https://raw.githubusercontent.com/salemprakash/EDA/main/Data/Employee%20Attrition.csv"

# 1. Load + identify numerical / categorical variables
df = pd.read_csv(URL)
numerical = df.select_dtypes(include="number").columns.tolist()
categorical = df.select_dtypes(exclude="number").columns.tolist()
print("1. Shape:", df.shape)
print("   Numerical variables  :", numerical)
print("   Categorical variables:", categorical, "\n")

# 2. Scatter plot - 3 variables (X, Y, colour)
plt.figure(figsize=(9, 6))
sns.scatterplot(data=df, x="Age", y="MonthlyIncome", hue="Attrition", alpha=0.7, palette="Set1")
plt.title("2. Age vs MonthlyIncome coloured by Attrition")
plt.tight_layout(); plt.show()

# 3. Scatter plot - 4 variables (X, Y, colour, shape)
plt.figure(figsize=(9, 6))
sns.scatterplot(data=df, x="Age", y="MonthlyIncome", hue="Attrition", style="Gender",
                alpha=0.7, palette="Set1")
plt.title("3. Age vs MonthlyIncome: colour = Attrition, shape = Gender")
plt.tight_layout(); plt.show()

# 4. Scatter plot - 4 variables (X, Y, colour, size = numeric)
plt.figure(figsize=(9, 6))
sns.scatterplot(data=df, x="Age", y="MonthlyIncome", hue="Attrition", size="YearsAtCompany",
                sizes=(20, 250), alpha=0.6, palette="Set1")
plt.title("4. Age vs MonthlyIncome: colour = Attrition, size = YearsAtCompany")
plt.legend(bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout(); plt.show()

# 5. Bubble plot - 3 numerical variables
sample = df.sample(250, random_state=1)           # sample keeps bubbles readable
plt.figure(figsize=(9, 6))
plt.scatter(sample["Age"], sample["MonthlyIncome"], s=sample["TotalWorkingYears"] * 12,
            alpha=0.5, color="dodgerblue", edgecolor="black")
plt.xlabel("Age"); plt.ylabel("MonthlyIncome")
plt.title("5. Bubble Plot: size = TotalWorkingYears")
plt.tight_layout(); plt.show()

# 6. facet_wrap equivalent
```

File: c:\Users\HP\OneDrive\Desktop\Computer_Science\EDA\CODES\qp5_multivariate.py

```
g = sns.relplot(data=df, x="Age", y="MonthlyIncome", col="JobRole", col_wrap=3,
                hue="Attribution", height=3, alpha=0.7, palette="Set1")
g.figure.suptitle("6. facet_wrap: Age vs MonthlyIncome by JobRole", y=1.02)
plt.show()

# 7. facet_grid equivalent (rows = Gender, columns = Department)
g = sns.relplot(data=df, x="Age", y="MonthlyIncome", row="Gender", col="Department",
                hue="Attrition", height=3.2, alpha=0.7, palette="Set1")
g.figure.suptitle("7. facet_grid: Gender (rows) x Department (cols)", y=1.02)
plt.show()
print("7. Observation: income rises with age in every panel; attrition (Yes) points cluster "
      "at lower incomes and younger ages, and the pattern is similar for both genders, "
      "with Sales showing the widest income spread.\n")

# 8. Pairs plot (4 numerical variables)
pair_vars = ["Age", "MonthlyIncome", "TotalWorkingYears", "YearsAtCompany"]
g = sns.pairplot(df, vars=pair_vars, hue="Attrition", corner=False, plot_kws={"alpha": 0.5},
                 palette="Set1")
g.figure.suptitle("8. Pairs plot", y=1.02)
plt.show()
print("8. Correlation matrix of selected variables\n", df[pair_vars].corr().round(2))
print("   Observation: MonthlyIncome, TotalWorkingYears, Age and YearsAtCompany are all "
      "positively correlated (income vs working years is the strongest, r ~ 0.77).\n")

# 9. Parallel coordinate plot (min-max scaled so all axes are comparable)
pc_vars = ["Age", "MonthlyIncome", "TotalWorkingYears", "YearsAtCompany", "DistanceFromHome"]
pc = df[pc_vars + ["Attrition"]].sample(300, random_state=2).copy()
pc[pc_vars] = (pc[pc_vars] - pc[pc_vars].min()) / (pc[pc_vars].max() - pc[pc_vars].min())
plt.figure(figsize=(10, 6))
parallel_coordinates(pc, "Attrition", color=["#1f77b4", "#d62728"], alpha=0.4)
plt.title("9. Parallel Coordinate Plot (scaled 0-1), grouped by Attrition")
plt.ylabel("Scaled value")
plt.tight_layout(); plt.show()

```

File: c:\Users\HP\OneDrive\Desktop\Computer_Science\EDA\CODES\qp6_time_series.py

```
#AirPassengers Dataset for Time Series Analysis

import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.arima.model import ARIMA

warnings.filterwarnings("ignore")

# 1. Load AirPassengers (monthly totals, Jan 1949 - Dec 1960)
values = [
    112, 118, 132, 129, 121, 135, 148, 148, 136, 119, 104, 118,
    115, 126, 141, 135, 125, 149, 170, 170, 158, 133, 114, 140,
    145, 150, 178, 163, 172, 178, 199, 199, 184, 162, 146, 166,
    171, 180, 193, 181, 183, 218, 230, 242, 209, 191, 172, 194,
    196, 196, 236, 235, 229, 243, 264, 272, 237, 211, 180, 201,
    204, 188, 235, 227, 234, 264, 302, 293, 259, 229, 203, 229,
    242, 233, 267, 269, 270, 315, 364, 347, 312, 274, 237, 278,
    284, 277, 317, 313, 318, 374, 413, 405, 355, 306, 271, 306,
    315, 301, 356, 348, 355, 422, 465, 467, 404, 347, 305, 336,
    340, 318, 362, 348, 363, 435, 491, 505, 404, 359, 310, 337,
    360, 342, 406, 396, 420, 472, 548, 559, 463, 407, 362, 405,
    417, 391, 419, 461, 472, 535, 622, 606, 508, 461, 390, 432,
]
AirPassengers = pd.Series(values, index=pd.date_range("1949-01", periods=144, freq="MS"),
                          name="AirPassengers")
print("1. AirPassengers loaded:", AirPassengers.shape, "\n")

# 2. First five and last five observations
print("2. First five\n", AirPassengers.head(5), "\n\n   Last five\n", AirPassengers.tail(5), "\n")

# 3. Start and end
print("3. Start:", AirPassengers.index.min().strftime("%b %Y"),
      "| End:", AirPassengers.index.max().strftime("%b %Y"), "\n")

# 4. Frequency and time-series characteristics
print("4. Frequency (observations per year): 12  (index freq =", AirPassengers.index.freqstr, ")")
print("   Type:", type(AirPassengers).__name__, "| dtype:", AirPassengers.dtype)
print("   Mean = %.2f, Std = %.2f, Min = %d, Max = %d\n" %
      (AirPassengers.mean(), AirPassengers.std(), AirPassengers.min(), AirPassengers.max()))

# 5. Monthly trend plot
plt.figure(figsize=(11, 5))
plt.plot(AirPassengers, color="navy")
plt.title("5. Monthly Airline Passengers (1949-1960)")
```

File: c:\Users\HP\OneDrive\Desktop\Computer_Science\EDA\CODES\qp6_time_series.py

```
plt.xlabel("Year"); plt.ylabel("Passengers (thousands)")
plt.grid(True, alpha=0.4)
plt.tight_layout(); plt.show()

# 6. Decompose into trend, seasonal, random
#    (multiplicative, because seasonal swings grow with the level of the series)
decomp = seasonal_decompose(AirPassengers, model="multiplicative", period=12)
print("6. Decomposition (first rows)\n",
      pd.DataFrame({"trend": decomp.trend, "seasonal": decomp.seasonal,
                    "random": decomp.resid}).head(12).round(3), "\n")

# 7. Plot the decomposed components
fig = decomp.plot()
fig.set_size_inches(11, 8)
fig.suptitle("7. Decomposition of AirPassengers (multiplicative)", y=1.01)
plt.tight_layout(); plt.show()

# 8. ACF and PACF
fig, ax = plt.subplots(1, 2, figsize=(13, 4))
plot_acf(AirPassengers, lags=36, ax=ax[0]);  ax[0].set_title("8a. ACF")
plot_pacf(AirPassengers, lags=36, ax=ax[1], method="ywm"); ax[1].set_title("8b. PACF")
plt.tight_layout(); plt.show()
print("8. ACF decays slowly (non-stationary, strong trend) with peaks at lags 12, 24, 36 "
      "(yearly seasonality). PACF cuts off after the first lags.\n")

# 9. Fit an ARIMA model  (seasonal ARIMA (2,1,1)(0,1,0)[12] on the raw series)
model = ARIMA(AirPassengers, order=(2, 1, 1), seasonal_order=(0, 1, 0, 12))
fit = model.fit()

# 10. Model summary + interpretation
print("10. ARIMA model summary")
print(fit.summary())
print("""
Interpretation:
 - order (2,1,1): 2 autoregressive terms, 1 non-seasonal difference, 1 moving-average term.
 - seasonal (0,1,0,12): one seasonal difference (lag 12) removes the yearly pattern.
 - Coefficients with small p-values (P>|z|) are significant; lower AIC/BIC = better model.
 - Ljung-Box p > 0.05 means residuals look like white noise (no leftover autocorrelation).
""")

# 11. Forecast the next 12 months
forecast = fit.get_forecast(steps=12)
pred = forecast.predicted_mean
conf = forecast.conf_int()
print("11. 12-month forecast\n", pred.round(1), "\n")

# 12. Actual vs forecast plot
plt.figure(figsize=(11, 5))
```

File: c:\Users\HP\OneDrive\Desktop\Computer_Science\EDA\CODES\qp6_time_series.py

```
97 | plt.plot(AirPassengers, label="Actual", color="navy")
98 | plt.plot(pred, label="Forecast", color="red", linestyle="--")
99 | plt.fill_between(conf.index, conf.iloc[:, 0], conf.iloc[:, 1], color="red", alpha=0.15,
100 |                  label="95% confidence interval")
101 | plt.title("12. AirPassengers: Actual vs 12-month ARIMA Forecast")
102 | plt.xlabel("Year"); plt.ylabel("Passengers (thousands)")
103 | plt.legend(); plt.grid(True, alpha=0.4)
104 | plt.tight_layout(); plt.show()
105 |
```

File: c:\Users\HP\OneDrive\Desktop\Computer_Science\EDA\CODES\qp7_1d_2d_nd_analysis.py

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

pd.set_option("display.width", 200)
pd.set_option("display.max_columns", 50)
sns.set_theme(style="whitegrid")
URL = "https://raw.githubusercontent.com/salemprakash/EDA/main/Data/Employee%20Attrition.csv"

# 1. Load dataset
df = pd.read_csv(URL)
print("1. Loaded:", df.shape, "\n")

# 2 & 3. Summary statistics stored in one structured data frame
num_vars = ["Age", "MonthlyIncome", "TotalWorkingYears", "YearsAtCompany"]
rows = {}
for c in num_vars:
    s = df[c]
    rows[c] = {
        "Mean": s.mean(), "Median": s.median(), "Minimum": s.min(), "Maximum": s.max(),
        "Range": s.max() - s.min(), "Variance": s.var(), "Std Dev": s.std(),
        "IQR": s.quantile(0.75) - s.quantile(0.25),
        "Skewness": stats.skew(s),              # e1071::skewness (type 3)
        "Kurtosis": stats.kurtosis(s),          # e1071::kurtosis (type 3, excess)
    }
summary_df = pd.DataFrame(rows).T.round(3)
print("2/3. Summary statistics\n", summary_df, "\n")

# 4. Frequency + relative frequency table for a categorical variable; pie + bar
freq = df["Department"].value_counts()
rel = df["Department"].value_counts(normalize=True).round(4)
print("4. Frequency / Relative frequency (Department)\n",
      pd.DataFrame({"Frequency": freq, "Relative Frequency": rel}), "\n")

fig, ax = plt.subplots(1, 2, figsize=(13, 5))
ax[0].pie(freq, labels=freq.index, autopct="%1.1f%%", startangle=90,
          colors=sns.color_palette("pastel"))
ax[0].set_title("4a. Pie Chart - Department")
sns.barplot(x=freq.index, y=freq.values, hue=freq.index, palette="pastel", ax=ax[1], legend=False)
ax[1].set_title("4b. Bar Plot - Department"); ax[1].set_ylabel("Frequency")
ax[1].tick_params(axis="x", rotation=15)
plt.tight_layout(); plt.show()

# 5. Frequency table with bin intervals; histogram + box plot
bins = pd.cut(df["MonthlyIncome"], bins=10)
print("5. Binned frequency table (MonthlyIncome)\n", bins.value_counts().sort_index(), "\n")
```

File: c:\Users\HP\OneDrive\Desktop\Computer_Science\EDA\CODES\qp7_1d_2d_nd_analysis.py

```
fig, ax = plt.subplots(1, 2, figsize=(13, 5))
ax[0].hist(df["MonthlyIncome"], bins=10, color="skyblue", edgecolor="black")
ax[0].set_title("5a. Histogram - MonthlyIncome"); ax[0].set_xlabel("MonthlyIncome")
sns.boxplot(y=df["MonthlyIncome"], ax=ax[1], color="lightcoral")
ax[1].set_title("5b. Box Plot (outliers = dots)")
plt.tight_layout(); plt.show()

# 6. Contingency table + row / column percentages
ct = pd.crosstab(df["Department"], df["Attrition"])
print("6. Contingency table\n", ct, "\n")
print("   Row percentages (%)\n", (ct.div(ct.sum(axis=1), axis=0) * 100).round(2), "\n")
print("   Column percentages (%)\n", (ct.div(ct.sum(axis=0), axis=1) * 100).round(2), "\n")

# 7. Stacked and grouped bar plots
fig, ax = plt.subplots(1, 2, figsize=(13, 5))
ct.plot(kind="bar", stacked=True, ax=ax[0], color=["#4c72b0", "#dd8452"])
ax[0].set_title("7a. Stacked Bar - Department vs Attrition"); ax[0].tick_params(axis="x", rotation=15)
ct.plot(kind="bar", ax=ax[1], color=["#4c72b0", "#dd8452"])
ax[1].set_title("7b. Grouped Bar - Department vs Attrition"); ax[1].tick_params(axis="x", rotation=15)
plt.tight_layout(); plt.show()

# 8. One categorical + one numerical: group statistics (aggregate())
agg = df.groupby("Department")["MonthlyIncome"].agg(["mean", "median", "std"]).round(2)
print("8. MonthlyIncome by Department\n", agg, "\n")

# 9. Side-by-side box plots + violin plot
fig, ax = plt.subplots(1, 2, figsize=(14, 5))
sns.boxplot(data=df, x="Department", y="MonthlyIncome", hue="Department", ax=ax[0],
            palette="Set2", legend=False)
ax[0].set_title("9a. Side-by-side Box Plots")
sns.violinplot(data=df, x="Department", y="MonthlyIncome", hue="Department", ax=ax[1],
               palette="Set2", legend=False)
ax[1].set_title("9b. Violin Plot")
plt.tight_layout(); plt.show()

# 10. Pearson & Spearman correlation + scatter with regression line
pearson = df["Age"].corr(df["MonthlyIncome"], method="pearson")
spearman = df["Age"].corr(df["MonthlyIncome"], method="spearman")
print(f"10. Age vs MonthlyIncome -> Pearson = {pearson:.3f}, Spearman = {spearman:.3f}\n")
plt.figure(figsize=(8, 6))
sns.regplot(data=df, x="Age", y="MonthlyIncome", scatter_kws={"alpha": 0.4},
            line_kws={"color": "red"})
plt.title("10. Scatter plot with Linear Regression Line")
plt.tight_layout(); plt.show()

# 11. Two categorical + one numerical
grp = df.groupby(["Gender", "Attrition"])["MonthlyIncome"].agg(["mean", "std"]).round(2)
print("11. MonthlyIncome by Gender x Attrition\n", grp, "\n")
```

File: c:\Users\HP\OneDrive\Desktop\Computer_Science\EDA\CODES\qp7_1d_2d_nd_analysis.py

```
97 |
98 | g = sns.catplot(data=df, x="Attrition", y="MonthlyIncome", col="Gender", kind="box",
99 |                 hue="Attrition", palette="Set2", height=4, legend=False)
100 | g.figure.suptitle("11a. Faceted Box Plot", y=1.03)
101 | plt.show()
102 |
103 | plt.figure(figsize=(8, 5))
104 | sns.barplot(data=df, x="Gender", y="MonthlyIncome", hue="Attrition", errorbar="sd",
105 |             capsize=0.1, palette="Set1")
106 | plt.title("11b. Grouped Bar Chart with Error Bars (mean +/- SD)")
107 | plt.tight_layout(); plt.show()
108 |
109 | # 12. One categorical + two numerical
110 | plt.figure(figsize=(9, 6))
111 | sns.scatterplot(data=df, x="Age", y="MonthlyIncome", hue="Attrition", style="Attrition",
112 |                 alpha=0.7, palette="Set1")
113 | plt.title("12. Age vs MonthlyIncome by Attrition")
114 | plt.tight_layout(); plt.show()
115 | print("12. Correlation (Age vs MonthlyIncome) separately for each Attrition level")
116 | for level, sub in df.groupby("Attrition"):
117 |     print(f"    {level}:\n", sub[["Age", "MonthlyIncome"]].corr().round(3), "\n")
118 |
119 | # 13. Correlation matrix + heatmap (>= 3 numerical features)
120 | hm_vars = ["Age", "MonthlyIncome", "TotalWorkingYears", "YearsAtCompany", "DistanceFromHome"]
121 | corr = df[hm_vars].corr()
122 | print("13. Correlation matrix\n", corr.round(3), "\n")
123 | plt.figure(figsize=(8, 6))
124 | sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", vmin=-1, vmax=1)
125 | plt.title("13. Correlation Heatmap")
126 | plt.tight_layout(); plt.show()
127 |
128 | # 14. Pairs plot / scatterplot matrix
129 | g = sns.pairplot(df[hm_vars + ["Attrition"]], hue="Attrition", diag_kind="kde",
130 |                  plot_kws={"alpha": 0.4}, palette="Set1")
131 | g.figure.suptitle("14. Pairs Plot", y=1.02)
132 | plt.show()
133 |
```

File: c:\Users\HP\OneDrive\Desktop\Computer_Science\EDA\CODES\qp8_clustering.py

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster
from scipy.spatial.distance import pdist

pd.set_option("display.width", 200)
pd.set_option("display.max_columns", 50)
URL = "https://raw.githubusercontent.com/salemprakash/EDA/main/Data/Employee%20Attrition.csv"

# PART A : K-MEANS CLUSTERING

print("\nK-MEANS CLUSTERING\n")

# 1. Load dataset
df = pd.read_csv(URL)
print("1. Loaded:", df.shape, "\n")

# 2. View dataset
print("2. Dataset preview\n", df.head(), "\n")

# 3. Missing values
print("3. Missing values:", int(df.isna().sum().sum()), "\n")

# 4. Dependent variable distribution (table)
print("4. Attrition distribution\n", df["Attrition"].value_counts(), "\n")

# 5. Numerical features only (others removed).
#    Zero-variance columns (EmployeeCount, StandardHours) carry no information
#    and cannot be standardised, so they are removed too.
X = df.select_dtypes(include="number")
X = X.loc[:, X.std() > 0]
print("5. Numerical features used:", X.shape[1], "\n   ", list(X.columns), "\n")

# 6. Standardise (scale)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print("6. Standardised data  mean ~", X_scaled.mean().round(3), " std ~", X_scaled.std().round(3), "\n")

# 7. K-Means with 2 centres
km2 = KMeans(n_clusters=2, n_init=25, random_state=42)
km2.fit(X_scaled)

# 8. Display K-Means results
print("8. K-Means (k=2) results")
print("   Cluster sizes:", np.bincount(km2.labels_))
```

File: c:\Users\HP\OneDrive\Desktop\Computer_Science\EDA\CODES\qp8_clustering.py

```
print("   Inertia (total within-cluster SS):", round(km2.inertia_, 2))
print("   First 10 assignments:", km2.labels_[:10])
print("   Cross-tab with Attrition\n", pd.crosstab(km2.labels_, df["Attrition"]), "\n")

# 9. Cluster centres
centers = pd.DataFrame(km2.cluster_centers_, columns=X.columns)
print("9. Cluster centres (standardised scale)\n", centers.round(3).T, "\n")

# 10. WCSS for K = 1 to 10
wcss = []
for k in range(1, 11):
    km = KMeans(n_clusters=k, n_init=25, random_state=42).fit(X_scaled)
    wcss.append(km.inertia_)
print("10. WCSS values")
for k, w in zip(range(1, 11), wcss):
    print(f"    K={k:2d}  WCSS={w:10.2f}")
print()

# 11. Elbow plot
plt.figure(figsize=(8, 5))
plt.plot(range(1, 11), wcss, marker="o", color="blue")
plt.title("11. Elbow Method"); plt.xlabel("Number of clusters (K)"); plt.ylabel("WCSS")
plt.xticks(range(1, 11)); plt.grid(True, alpha=0.4)
plt.show()

# 12. K-Means at the elbow recommendation
ks = np.arange(1, 11)
p1, p2 = np.array([1, wcss[0]]), np.array([10, wcss[-1]])
pts = np.column_stack([ks, wcss])
v = p2 - p1
dist = np.abs(v[0] * (pts[:, 1] - p1[1]) - v[1] * (pts[:, 0] - p1[0])) / np.linalg.norm(v)
best_k = int(ks[dist.argmax()])
print("12. Elbow recommendation: K =", best_k)

km_best = KMeans(n_clusters=best_k, n_init=25, random_state=42).fit(X_scaled)
print("    Cluster sizes:", np.bincount(km_best.labels_))
print("    Cross-tab with Attrition\n", pd.crosstab(km_best.labels_, df["Attrition"]), "\n")

# Visualise the clusters on the first two features
plt.figure(figsize=(8, 6))
plt.scatter(X_scaled[:, 0], X_scaled[:, X.columns.get_loc("MonthlyIncome")],
            c=km_best.labels_, cmap="viridis", alpha=0.6)
plt.xlabel("Age (scaled)"); plt.ylabel("MonthlyIncome (scaled)")
plt.title(f"12. K-Means clusters (K = {best_k})")
plt.show()

# PART B : HIERARCHICAL CLUSTERING

```

File: c:\Users\HP\OneDrive\Desktop\Computer_Science\EDA\CODES\qp8_clustering.py

```
print("\nHIERARCHICAL CLUSTERING\n")

# 1. Load dataset
hdf = pd.read_csv(URL)
print("1. Loaded:", hdf.shape, "\n")

# 2. View dataset
print("2. Dataset preview\n", hdf.head(), "\n")

# 3. Missing values
print("3. Missing values:", int(hdf.isna().sum().sum()), "\n")

# 4. Numerical attributes only
HX = hdf.select_dtypes(include="number")
HX = HX.loc[:, HX.std() > 0]
print("4. Numerical attributes:", HX.shape[1], "\n")

# 5. Standardise
HX_scaled = StandardScaler().fit_transform(HX)

# For readability and speed the dendrogram is built on a random sample of 150 employees
rng = np.random.default_rng(1)
idx = rng.choice(len(HX_scaled), 150, replace=False)
H = HX_scaled[idx]
labels_sample = hdf["EmployeeNumber"].values[idx]
print("5. Standardised; using a random sample of", len(idx), "employees for the dendrogram\n")

# 6. Euclidean distance
d = pdist(H, metric="euclidean")
print("6. Euclidean distances computed:", len(d), "pairs\n")

# 7. Hierarchical clustering with Ward's method
Z = linkage(H, method="ward", metric="euclidean")
print("7. Ward linkage matrix computed:", Z.shape, "\n")

# 8. Dendrogram
plt.figure(figsize=(14, 6))
dendrogram(Z, labels=labels_sample, leaf_rotation=90, leaf_font_size=6)
plt.title("8. Dendrogram (Ward's method)"); plt.xlabel("EmployeeNumber"); plt.ylabel("Distance")
plt.tight_layout(); plt.show()

# 9. Cut into 3 clusters + assignments
clusters = fcluster(Z, t=3, criterion="maxclust")
assign = pd.DataFrame({"EmployeeNumber": labels_sample, "Cluster": clusters})
print("9. Cluster assignments (first 15)\n", assign.head(15), "\n")

# 10. Number of samples per cluster
print("10. Samples in each cluster\n", pd.Series(clusters).value_counts().sort_index())
```

File: c:\Users\HP\OneDrive\Desktop\Computer_Science\EDA\CODES\qp8_clustering.py

145 |

File: c:\Users\HP\OneDrive\Desktop\Computer_Science\EDA\CODES\qp9_pca.py

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

try:
    from adjustText import adjust_text
except ImportError:
    adjust_text = None
    print("Tip: run  pip install adjustText  for automatically non-overlapping labels.\n")

pd.set_option("display.width", 200)
pd.set_option("display.max_columns", 50)

URL = "https://raw.githubusercontent.com/salemprakash/EDA/main/Data/Employee%20Attrition.csv"
REG_NO = "23BDS0287"

# 1. Load dataset (libraries pandas, numpy, matplotlib, sklearn are imported above)
df = pd.read_csv(URL)
print("1. Dataset loaded; packages imported (pandas, numpy, matplotlib, scikit-learn).\n")

# 2. Rows and columns
print(f"2. Total rows = {df.shape[0]}, total columns = {df.shape[1]}\n")

# 3. Numeric columns -> clean -> normalise  (data_scaled)
num = df.select_dtypes(include="number").dropna()
num = num.loc[:, num.std() > 0]
data_scaled = pd.DataFrame(StandardScaler().fit_transform(num), columns=num.columns)
print("3. data_scaled shape:", data_scaled.shape, "\n")

# 4. PCA + summary
pca = PCA()
scores = pca.fit_transform(data_scaled)
n_pc = scores.shape[1]
pc_names = [f"PC{i}" for i in range(1, n_pc + 1)]
summary = pd.DataFrame({
    "Standard deviation": np.sqrt(pca.explained_variance_),
    "Proportion of Variance": pca.explained_variance_ratio_,
    "Cumulative Proportion": np.cumsum(pca.explained_variance_ratio_),
}, index=pc_names).round(4)
print("4. PCA summary\n", summary, "\n")

# 5. Scree plot with data labels, y-axis 0-60
var_pct = pca.explained_variance_ratio_ * 100
plt.figure(figsize=(11, 5))
plt.plot(range(1, n_pc + 1), var_pct, marker="o", color="steelblue")
for i, v in enumerate(var_pct, start=1):
```

File: c:\Users\HP\OneDrive\Desktop\Computer_Science\EDA\CODES\qp9_pca.py

```
    plt.text(i, v + 1, f"{v:.1f}", ha="center", fontsize=8)
plt.ylim(0, 60)
plt.xticks(range(1, n_pc + 1))
plt.title("5. Scree Plot"); plt.xlabel("Principal Component"); plt.ylabel("Percentage of Variance Explained")
plt.grid(True, alpha=0.4)
plt.tight_layout(); plt.show()

# 6. PCA biplot (observations + variable loading arrows, non-overlapping labels)
loadings = pca.components_[:2].T                        # (variables x 2)
scale = np.abs(scores[:, :2]).max() * 0.8
fig, ax = plt.subplots(figsize=(11, 9))
ax.scatter(scores[:, 0], scores[:, 1], s=8, alpha=0.25, color="grey")
texts = []
for i, var in enumerate(num.columns):
    x, y = loadings[i, 0] * scale, loadings[i, 1] * scale
    ax.arrow(0, 0, x, y, color="red", alpha=0.7, head_width=0.08, length_includes_head=True)
    texts.append(ax.text(x, y, var, color="darkred", fontsize=9))
if adjust_text:
    adjust_text(texts, ax=ax, arrowprops=dict(arrowstyle="-", color="darkred", lw=0.5))
ax.axhline(0, color="black", lw=0.5); ax.axvline(0, color="black", lw=0.5)
ax.set_xlabel(f"PC1 ({var_pct[0]:.1f}%)"); ax.set_ylabel(f"PC2 ({var_pct[1]:.1f}%)")
ax.set_title("6. PCA Biplot - Employee Attrition Data")
plt.tight_layout(); plt.show()

# 7. PCA scores -> data frame pca_scores, first six rows
pca_scores = pd.DataFrame(scores, columns=pc_names)
print("7. First six rows of pca_scores\n", pca_scores.head(6), "\n")

# 8. Scatter plot PC1 vs PC2 : blue circular points, registration number in title
plt.figure(figsize=(8, 6))
plt.scatter(pca_scores["PC1"], pca_scores["PC2"], color="blue", marker="o", alpha=0.6)
plt.title(f"8. PC1 vs PC2 - {REG_NO}")
plt.xlabel("PC1"); plt.ylabel("PC2")
plt.grid(True, alpha=0.4)
plt.tight_layout(); plt.show()

```

File: c:\Users\HP\OneDrive\Desktop\Computer_Science\EDA\CODES\requirements.txt

```
pandas
numpy
matplotlib
seaborn
scipy
scikit-learn
statsmodels
adjustText
```

Table of Contents

==========

qp1_merging_transformation.py 3
qp2_import_transform_stats.py 5
qp3_cleaning_univariate.py 8
qp4_bivariate.py 13
qp5_multivariate.py 16
qp6_time_series.py 18
qp7_1d_2d_nd_analysis.py 21
qp8_clustering.py 24
qp9_pca.py 28
requirements.txt 30

PDF of project created on 10/19/2026 with All to PDF - Ai Facilitator extension for VSCode